CrewAI实战解析：程序1题目生成器

- 来源: https://class.imooc.com/lesson/5293#mid=105159
- 阶段: 阶段六: 【智能体】从单体到多体
- 周: 第12周 Multi-Agent：多智能体协作
- 模块: CrewAI 实战
- 课节: 图文 1-3 CrewAI实战解析：程序1题目生成器

1-3 CrewAI实战解析：程序1题目生成器

CrewAI实战解析：程序1题目生成器

> [!TIP] 阅读指南 上一节我们了解了系统概览和学习目标。本节重点讲解程序1（question_generator.py），通过它深入理解CrewAI三大核心组件：Crew、Agent、Task的协作关系。

业务流程回顾

程序1实现了从教学大纲到题库的完整流程：

```
──────────────────────────────────────────
  阶段1：设计题目框架
  • AI决策：分析大纲，规划题目结构
  • 输出：题目设计方案
──────────────────────────────────────────
  阶段2：生成具体题目
  • AI决策：生成题目内容和选项
  • 输出：JSON格式题目
──────────────────────────────────────────
  阶段3：数据持久化
  • 传统业务：保存到 data/questions.json
──────────────────────────────────────────
```

CrewAI三大组件的角色分工

在CrewAI框架中，Crew、Agent、Task各司其职，形成了清晰的三层协作结构：

三大组件的职责：

三大组件的职责

组件	类比角色	职责	程序1实例
Crew	项目经理	统筹全局，决定执行模式	ExerciseCrew（Sequential模式）
Agent	专业员工	具备专业能力，执行任务	题目生成专家（QuestionExpert）
Task	具体工作	明确目标和输出要求	Task1: 设计框架 Task2: 生成题目

用团队协作来理解三者关系：

用团队协作来理解三者关系

想象一个软件公司的项目团队：

```
ExerciseCrew（团队）
  ├─ 角色：项目经理
  ├─ 持有资源：题目生成专家（员工）、LLM（工具）
  └─ 核心职责：
      • 创建工单（Task1、Task2）
      • 分配给员工（Agent）
      • 决定执行顺序（Sequential模式）
      • 管理工单流转（Task1输出 → Task2输入）

QuestionExpert（员工）
  ├─ 角色：Python教育专家
  ├─ 技能：编程教学、题目设计
  └─ 工作方式：
      • 接收工单（Task）
      • 调用工具（LLM）完成工作
      • 提交成果（输出结果）

Task1 & Task2（工单）
  ├─ Task1工单内容：
  │   • 任务：分析大纲，设计题目框架
  │   • 指派给：QuestionExpert
  │   • 要求：输出Markdown设计方案
  │
  └─ Task2工单内容：
      • 任务：根据Task1方案生成具体题目
      • 指派给：QuestionExpert
      • 依赖：必须等Task1完成
      • 要求：输出JSON格式题目
```

在程序1中的实际运作：

在程序1中的实际运作

- ExerciseCrew创建工单： task1 = create_design_task(self.question_expert, syllabus) task2 = create_generate_task(self.question_expert, task1.output) 1 2 代码块 项目经理说： “我需要两个工单，Task1设计框架，Task2生成题目”

ExerciseCrew分配工单给员工：

ExerciseCrew分配工单给员工

```
crew = Crew(
    agents=[self.question_expert],  # 员工名单
    tasks=[task1, task2],            # 工单列表
    process=Process.sequential       # 工单顺序：先Task1，再Task2
)
```

项目经理说：“QuestionExpert，这两个工单都给你，按顺序做”

- QuestionExpert执行工单： 拿到Task1工单 → 调用LLM → 输出设计方案 拿到Task2工单 → 使用Task1结果 → 调用LLM → 输出JSON题目
- ExerciseCrew收集成果： result = crew.kickoff() # 启动团队工作，等待完成 1 代码块 项目经理说： “开始干活！等你们都做完我收成果”

关键理解：

关键理解

- Crew不干活：它只负责协调，不直接调用LLM

Crew不干活

- Agent干活：真正调用LLM的是Agent

Agent干活

- Task是指令：Task里的description就是给Agent的工作说明书

```
description
```

Task是指令

- Sequential保证顺序：Crew确保Task2等Task1完成后才开始

Sequential保证顺序

代码实现：从组件到协作

让我们看看这三个组件如何在代码中协同工作。

步骤1：定义Agent（专业员工）

```
# agents/question_expert.py

def create_question_expert(llm) -> Agent:
    """创建题目生成专家Agent"""
    return Agent(
        role="Python编程练习题生成专家",
        goal="根据教学大纲设计高质量的Python编程练习题",
        backstory="""
        你是一位经验丰富的Python教育专家，深刻理解初学者的学习路径。
        你擅长将复杂概念转化为清晰的练习题，帮助学生循序渐进地掌握知识。
        """,
        llm=llm,
        verbose=True
    )
```

关键点：

关键点

- role: Agent的身份定位
- goal: Agent的工作目标
- backstory: Agent的专业背景（影响决策风格）
- llm: Agent使用的语言模型

步骤2：定义Task（具体工作）

```python
# tasks/question_tasks.py

def create_design_task(agent: Agent, syllabus: str) -> Task:
    """创建Task1：设计题目框架"""
    return Task(
        description=f"""
        基于以下教学大纲，设计练习题的整体框架：

        {syllabus}

        要求：
        1. 分析大纲的知识点层次
        2. 规划题目的难度梯度
        3. 设计题目类型分布
        """,
        expected_output="题目框架设计方案（Markdown格式）",
        agent=agent  # 指定由哪个Agent执行
    )

def create_generate_task(agent: Agent, design_result: str) -> Task:
    """创建Task2：生成具体题目（依赖Task1输出）"""
    return Task(
        description=f"""
        根据以下设计方案，生成5道选择题：

        {design_result}

        要求：
        1. 严格遵循JSON格式
        2. 每题4个选项（A/B/C/D）
        3. 包含详细解析
        """,
        expected_output="题目列表（JSON格式）",
        agent=agent,
        context=[design_result]  # 依赖Task1的输出
    )
```

- description: 任务的详细说明（这是给Agent看的Prompt）
- expected_output: 明确输出格式
- agent: 绑定执行者
- context: 任务间依赖关系（Task2需要Task1的结果）

步骤3：Crew协调执行（项目经理）

```
# crews/exercise_crew.py

class ExerciseCrew:
    """统一的Crew，负责协调所有阶段"""

    def __init__(self, llm):
        self.llm = llm
        self.question_expert = create_question_expert(llm)

    def run_question_generation(self, syllabus: str) -> str:
        """
        程序1调用：执行题目生成流程（Sequential模式）
        """
        # 创建两个Task
        task1 = create_design_task(self.question_expert, syllabus)
        task2 = create_generate_task(self.question_expert, task1.output)

        # 创建Crew并执行
        crew = Crew(
            agents=[self.question_expert],
            tasks=[task1, task2],
            process=Process.sequential,  # Sequential模式
            verbose=True
        )

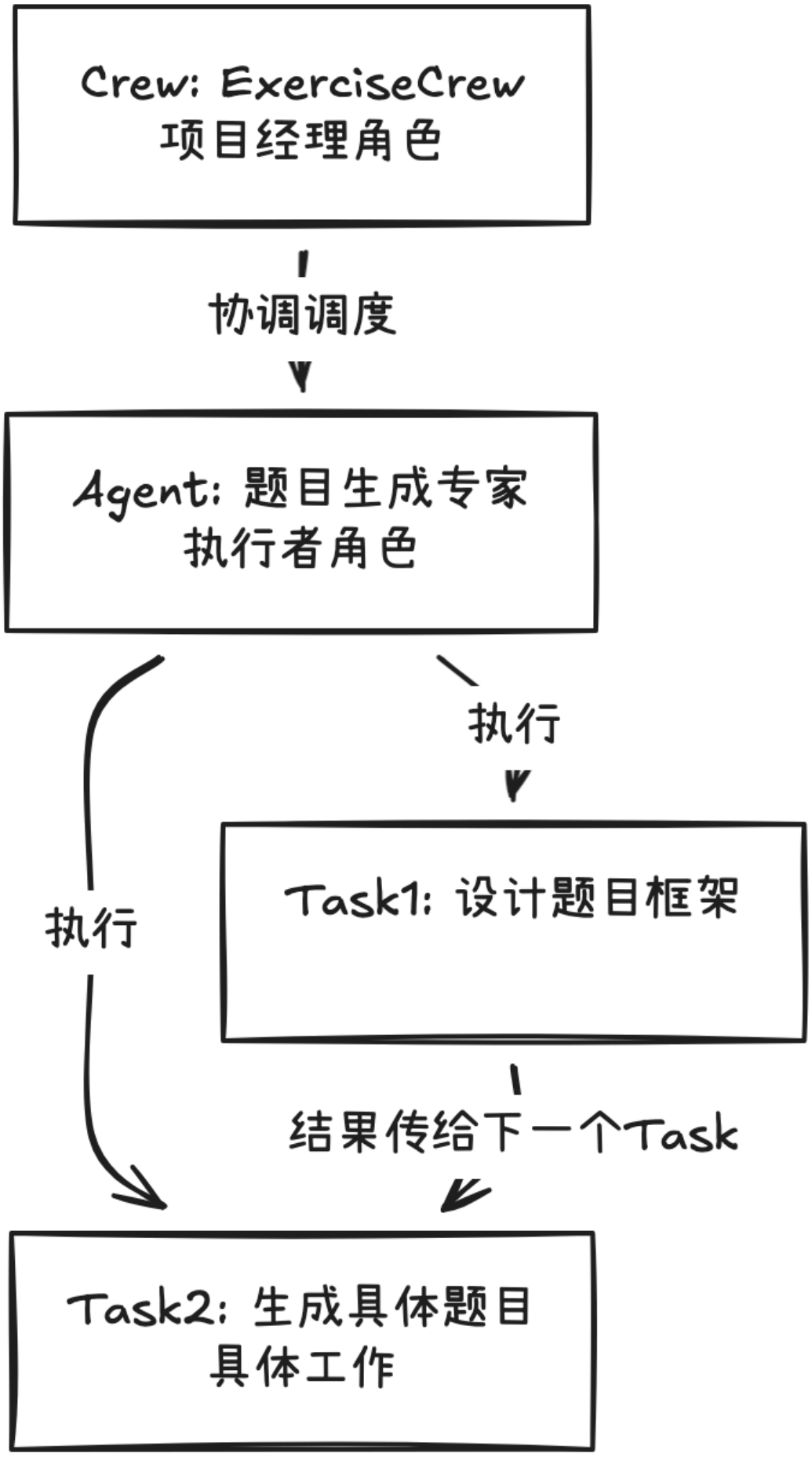
        result = crew.kickoff()
        return result
```

- agents: 参与的Agent列表（程序1只需1个Agent）
- tasks: 任务列表（按执行顺序排列）
- process=Process.sequential: 指定执行模式
- kickoff(): 启动执行

下一节预告

下一节我们将学习程序2，看看Hierarchical模式如何协调多个Agent并行工作，以及如何将传统代码与AI决策完美融合。

图片


Crew: ExerciseCrew
项目经理角色
协调调度
Agent: 题目生成专家
执行者角色
执行
执行
Task1: 设计题目框架
结果传给下一个Task
Task2: 生成具体题目
具体工作